Day35 & Day36 作业

银行 GraphRAG 与混合检索

Neo4j GDS	LangGraph	Milvus 2.6
Dijkstra · PageRank · Louvain	多路召回 · RRF · 硬规则	Dense · BM25 · Hybrid

姓名：待填写 班级：AI03 日期：2026-08-30

完整代码、测试、复现手册和真实集成证据随文档提交

姓名：待填写 班级：AI03 日期：2026-08-30

> 说明：本文实验数据均为虚构教学数据。完整源码、Cypher、SQL、测试与验收记录随文档一并提交。

本文按原题顺序覆盖：Dijkstra 与 PageRank 总结、Louvain 实践、GraphRAG 三项扩展、Milvus 混合检索、10 道面试题、Cypher 全练习、NL2SQL 和 NL2Cypher。

- 图算法：真实 Neo4j 5.26 + GDS 2.13.12 + APOC 5.26.30。
- 工作流：LangGraph 1.2.11，五个节点和显式状态。
- 向量库：真实 Milvus 2.6.17，Dense + BM25 + RRF。
- 金融规则：风险、起购金额和产品状态作为不可被模型覆盖的硬约束。
- 查询安全：只读、白名单、参数化、单语句、自动 LIMIT。
- 自动验证：27 项测试、静态检查、编译和两套真实数据库集成。
- 可复现性：提供 PowerShell 启停脚本、完整复现手册和错误记录。
- 能力边界：AI 验证不等于个人掌握，闭卷训练另附。

一、Dijkstra 与 PageRank 学习总结

1. Dijkstra 最短路径

Dijkstra 用于在“边权非负”的图中计算从一个起点到一个或多个终点的最低总成本路径。算法维护每个节点的当前最小已知距离，每轮确认距离最小的未处理节点，再用它更新相邻节点，直到到达目标。

本实验创建 A、B、C、D 四个车站：A→B 权重1，B→D权重2，A→C权重4，C→D权重1。候选路径 A→B→D 的总成本为3，A→C→D为5，因此结果是：

```
totalCost = 3.0
path = [A站, B站, D站]
```

银行场景中可把边权解释成转账成本、时间、风险或手续费，用于资金路径、服务网点路线、关系链最小代价等问题。限制是不能直接处理负权边，且最短不等于业务最合规。

2. PageRank 页面排名

PageRank 衡量节点在链接网络中的结构重要性。被重要节点指向的节点会得到更高贡献，因此它不是简单统计入边数量。本实验把产品共同购买关系投影成产品图，运行 `gds.pageRank.stream`，当前固定样例排名为 P002、P001、P003、P005。

PageRank 可用于发现核心产品、关键客户或交易网络中心节点；但 PageRank 高只表示“图结构重要”，不能替代客户适当性校验。

3. GDS 的通用执行流程

1. Neo4j 中保存业务节点和关系；
2. `gds.graph.project` 创建内存计算图；
3. 用 `stream` 模式运行算法并查看结果；
4. 结果满足预期后，生产场景才考虑 `write` 回写；
5. 用 `gds.graph.drop` 清理不再使用的内存图。

本实验选择 `stream`，算法结果不会写回数据库，便于重复验收。

二、Louvain 社区发现实践

Louvain 通过最大化模块度，让社区内部连接更密、社区之间连接更疏。本实验把客户 `FOLLOWS` 关系按无向、带 `strength` 权重投影，然后执行：

```
CALL gds.louvain.stream(
  'day3536_social',
  {relationshipWeightProperty:'strength'}
)
YIELD nodeId, communityId
```

真实运行结果：C001 与 C002 同社区，C003 与 C004 同社区，孤立的 C005 单独成社区；社区数不小于2，模块度大于0，自动断言通过。

社区编号只是分组 ID，不是客户等级。推荐系统可以统计目标客户所在社区的产品偏好，但最终仍需风险、金额和产品状态过滤。

三、LangGraph + Neo4j GraphRAG

1. 目标

仅用文本相似度会遗漏关系证据；仅用图关系又无法理解用户自然语言。因此系统使用三路召回：

```
用户与问题
 -> 图关系召回
 -> 社区召回
 -> 向量召回
 -> RRF融合
 -> 金融硬规则过滤
 -> 推荐与解释
```

2. LangGraph 状态和节点

`RecommendationState` 保存客户、问题、三路结果、融合结果和最终结果。工作流包含五个业务节点：

1. `graph_retrieve`：查找与目标客户有共同购买证据的其他客户产品；
2. `community_retrieve`：统计同社区客户购买的产品；
3. `vector_retrieve`：查询文本与产品描述做语义相似；
4. `rrf_fuse`：按名次融合三路结果；
5. `business_filter`：过滤停售、风险超限和金额不足产品。

代码位于 `src/day3536_homework/workflow.py`，通过 `StateGraph.add_node`、`add_edge`、`compile` 和 `invoke` 构建执行。

3. 额外任务1：社区检测检索

`InMemoryGraphRetriever.community_retrieve` 读取客户 `community_id`，只统计同社区、非本人、且目标客户尚未购买的产品，结果附带“同社区客户购买热度”解释。真实 Louvain 结果可通过 Neo4j 适配器替换内存社区字段。

4. 额外任务2：可解释性模块

每个 `RetrievalHit` 保存：

- `sources`：来自 graph、community、vector 或 Milvus 两路；
- `explanation`：共同购买见证、社区热度或语义与硬规则说明；
- `score`：融合后的排序分数。

示例：P005 的解释为“张明与李芳都购买过稳健理财；后者还购买了灵活存款”。这比只输出一个相似度更容易审查。

5. 额外任务3：RAG 方法对比

方法	优势	盲区	本实验用途
向量检索	能理解同义和口语	精确代码/关系较弱	用户需求对产品描述
BM25	精确关键词强、可解释	同义泛化弱	“灵活、稳健”等词
图关系	路径和多跳证据清楚	依赖图覆盖	相似客户共同购买
社区检索	发现群体偏好	社区可能漂移	同群客户热度
混合检索	多路互补	成本、调参、评估更复杂	RRF统一融合

固定评估集位于 `evaluation.py`。实验结论不是“混合永远更好”，而是必须按查询类型比较 Precision@K、Recall@K、MRR、延迟和硬规则违规率。

四、Milvus 迁移与混合检索优化

1. 为什么从 Neo4j 向量索引扩展到 Milvus

Neo4j 适合关系遍历和图算法；Milvus 专注大规模向量索引、多向量字段、过滤和混合检索。合理架构是职责互补，而不是用一个数据库替代所有数据库。

2. Schema 与索引

Collection：`day3536_bank_products_hybrid`。字段包括：产品ID、名称、描述、4维稠密向量、BM25稀疏向量、风险分、起购金额、有效状态。

- `dense_vector`：`FLOAT_VECTOR`，COSINE + AUTOINDEX；
- `description`：开启 analyzer；
- `sparse_vector`：由 `FunctionType.BM25` 自动生成；
- 稀疏索引：`SPARSE_INVERTED_INDEX` + BM25。

3. 两路检索和 RRF

系统创建两个 `AnnSearchRequest`：一个查 `dense_vector`，一个用原始文本查 `sparse_vector`，再通过 `RRFRanker(60)` 合并。检索表达式提前限制：

```
active == true
and risk_score <= 客户风险分
and min_investment <= 客户可用金额
```

真实 Milvus 2.6.17 验收：插入5条，最终返回 `P005、P001、P002`；P003 风险过高、P004 起购金额过高，均未返回。

4. 本次发现的真实边界

第一次插入后立即检索未满足断言，随后查询数据已可见，根因是写入和检索可见性的时序。修复是在 `insert_products()` 后显式 `flush()`，再完整回归。该失败已经写入验收记录。

五、RAG 混合检索 10 道面试题

1. 什么是混合检索，为什么不只用向量检索?

混合检索组合稠密语义、稀疏关键词和图关系等通道。向量擅长语义，但对产品编号、精确数字和专有名词可能不稳定；BM25 擅长精确词，图检索能提供关系路径。组合能提高覆盖，但必须用固定评估集证明收益。

2. BM25 与稠密向量各自擅长什么?

BM25 根据词频、逆文档频率和文档长度评分，擅长关键词；稠密向量由 Embedding 产生，擅长同义表达和上下文语义。产品代码、法规术语偏 BM25，口语需求偏稠密向量。

3. RRF 和加权分数融合有什么区别?

RRF 只用名次，避免不同通道分数量纲不一致；加权融合先归一化，再按业务重要性加权，控制更强但需要可靠评估集调参。缺少标注数据时，RRF 是稳健基线。

4. GraphRAG 与普通 RAG 的差别?

普通 RAG 主要检索文本块；GraphRAG 还利用实体、关系、多跳路径、社区和图算法，更适合“谁与谁有什么关系”“相似客户还买了什么”。代价是实体消歧、图更新和查询复杂度更高。

5. 怎样减少 RAG 幻觉?

使用可信数据源、检索阈值、引用证据、拒答规则、结构化输出和固定评估集；生成前先做权限与业务过滤，生成后检查引用是否真的支持结论。不能只靠提示词要求“不要幻觉”。

6. 怎样评估检索系统?

离线使用带相关性标注的问题集，计算 Precision@K、Recall@K、MRR、nDCG，同时监控硬规则违规率和零结果率。线上可观察点击和转化，但金融场景不能只优化点击率，合规是单项硬门槛。

7. Chunk 大小怎样选择?

小块定位精确但上下文不全，大块上下文完整但噪声高。应按标题和语义结构切分，保留来源，再用评估集比较大小和重叠；知识图谱关系不必机械切成文本块。

8. 检索结果很多，怎样优化上下文?

先召回较多候选，再去重、业务过滤、重排、相邻块扩展并按 token 预算裁剪；把来源、时间和权限做成元数据过滤。长上下文不是越多越好，噪声会稀释关键证据。

9. 金融推荐中的过滤应放在哪里?

最好两层防护：检索时用标量表达式提前过滤，融合后再用确定性函数复核。风险等级、起购金额和停售状态是硬约束，不能让模型或相似度覆盖。

10. 线上 RAG 延迟高怎样排查?

先分段计时 Embedding、每路召回、图查询、重排和模型生成，再检查索引、过滤选择性、top_k、连接池、批处理和缓存，并记录 P50/P95/P99。不要先盲目换大模型。

六、Cypher 全部练习

完整可运行答案位于：

- `cypher/01_初始化银行图数据.cypher`
- `cypher/02_Cypher全部练习答案.cypher`
- `cypher/03_Dijkstra_PageRank_Louvain.cypher`
- `cypher/04_验收断言.cypher`

覆盖基础查询、条件过滤、幂等创建、更新/删除候选、多跳关系、聚合、`OPTIONAL MATCH`、`CALL` 子查询、APOC Map、APOC 日期、Dijkstra、PageRank、Louvain 和 Node Similarity。为保护验收数据，原删除练习改为“返回将被删除的候选”，未执行破坏性删除。

七、NL2SQL 模块

流程：自然语言 → 规则/模型生成 SQL → SQLGlot AST 解析 → 单语句和只读检查 → 表白名单 → 自动 LIMIT → SQL 只读执行器。

本实验包含4个问题：稳健用户数、各城市购买总额、购买P001的用户、各风险等级平均收益。4条查询已在真实 SQLite 引擎上执行，MySQL 初始化版在 `sql/` 目录提供。

安全点：

- 拒绝 INSERT、UPDATE、DELETE、DROP、ALTER、CREATE、MERGE 等；
- 只允许 users、products、transactions；
- 默认补 `LIMIT 100`；
- 生成器和执行器职责分离。

八、NL2Cypher 模块

流程与 NL2SQL 类似，但增加 Cypher 专用限制：只允许单条 `MATCH ... RETURN`，禁止写入/管理命令和模型直接 `CALL` 过程，标签白名单，默认 LIMIT。

涉及客户或产品ID时使用参数化：

```
MATCH (c:Customer {customer_id: $customer_id})-[:PURCHASED]->(p:Product)
RETURN p.product_id, p.name
LIMIT 100
```

参数字典独立传入 `{"customer_id":"C001"}`。真实 Neo4j 验收返回 P001 和 P002。

九、测试与结论

确定性测试：27项通过；Ruff 静态检查通过；Python 编译通过。真实 Neo4j/APOC/GDS、Dijkstra、PageRank、Louvain、NL2Cypher 全部通过；真实 Milvus 稠密+BM25+RRF 通过。

本实验的核心结论是：推荐系统不能只追求相关性。正确顺序是多路召回、可比较的融合、确定性金融适配、证据解释和自动回归；模型负责理解和生成候选，规则与测试负责守住边界。

个人运行证据（提交前由本人填写）

- [ ] 已运行 `uv run python -m pytest -q`，结果：_____。
- [ ] 已运行 Neo4j 全部脚本，Dijkstra 路径：_____。
- [ ] 已观察 PageRank 第一名：_____。
- [ ] 已观察 Louvain 社区分组：_____。
- [ ] 已运行 Milvus 混合检索，返回产品：_____。
- [ ] 已把无隐私信息的运行截图插入提交附件。

个人口述复盘（提交前由本人填写）

1. 三路召回的输入、处理和输出：_____。
2. RRF 不直接相加原始分数的原因：_____。
3. 金融硬规则必须放在确定性代码中的原因：_____。
4. 本次遇到的一个错误、根因和修复：_____。

本人确认：文档中的运行结果已在个人环境复核；能够说明核心流程与安全边界。 签名：_____ 日期：_____